

GLIMMER-CS-EASY-02:指针

step1.指针与结构体

1.什么是指针

(1)

- 一般地，指针的定义方式为：int *a = &x; (大概率是没有二般地，反正我没查到，当然变式还是有很多的，在此不一一列举)
- 指针变量的大小是固定的，其大小取决于系统架构（即操作系统的位数，有32位系统和64位系统，分别对应4字节和8字节，说白了，看CPU性能），与指向类型、数据大小无关

(2)

- 输出结果：20 10 （意思是x = 20; y = 10;)
- 原因：
 - int* p = &x; 将x的地址交给了p，使得指针p与x建立了联系，通过*p = 20修改了p所指向的变量x的值（即x变为20）
 - 数组名相当于指针，指向的是数组的第一个元素（即arr[0]），而 * arr理应就指称数组中的第一个数（即3）。在这里，首先要声明：++a的返回值是a+1，副作用则是将a+1（而a++的返回值是a，副作用同前）那么 int y = ++*arr + *++q; 则可以解释为：y得到了数组arr中第一个值加一与第二个值之和（长难句(bushi)），即4+6=10

(3)

- 野指针：在指针变量声明之后没有进行初始化或者已释放内存的指针，叫做野指针
- 顾名思义，通过野指针访问内存是不可预知的，很容易导致数据错乱和程序崩溃。故而，我们需要在定义指针的同时对其进行初始化，如果暂时没有需要存放的地址可以先赋值为空指针（如int* a = 0;或者int* a = NULL;)
- 在上一题里我就已经将swap()修改好了，这里重新copy一下吧：

```
1  #include <stdio.h>
2
3  void swap(int *a, int *b){
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8
9  int main(){
10     int a = 10;
11     int b = 20;
12     swap(&a, &b);
13
14     printf("a=%d,b=%d",a,b);
15     return 0;
16 }
```

有图有真相：

```
getNew > getNew2 > C swap.c > swap(int *, int *)
1  #include <stdio.h>
2
3  void swap(int *a, int *b){
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8
9  int main(){
10     int a = 10;
11     int b = 20;
12     swap(&a, &b);
13
14     printf("a=%d,b=%d",a,b);
15     return 0;
16 }
```

问题 输出 调试控制台 终端 端口

...
eter=mi'
a=20,b=10
PS D:\c\.vscode>

行 6, 列 13 空格: 2 UTF-8 CRLF { } C

2.什么是结构体

(1)

结构体定义如下：

```
1  typedef struct PerInfo{
2      char name[10];
3      char sex;
4      int age;
5      double height;
6  } self_information;
```

(2) 结构体指针

结构体指针即指向结构体的指针，与普通指针的定义方式有些类似：

```
1  struct PerInfo stu = {"mark", 'M', 114514, 2.15}; //单字符要用单引号
2  struct PerInfo *p = &stu; //分开写也是可以的
```

函数之间只存在值的传递，而通过指针p就可以访问结构变量stu，从而在不同函数之间建立起联系，并且我们还可以通过结构体指针直接访问结构变量中的变量，如：p->age = 18（这等价于 (*p).age = 18）

当然结构体指针还可用于申请动态内存，在此就不做展开了

(3) 内存对齐规则

内存对齐规则：数据在内存中的起始地址必须是某个特定数值（对齐值）的整数倍（想了一下没什么可改的，直接Ctrl+C了）

基本数据类型的对齐规则（没问，但是我觉得有必要写一下，对比学习嘛）

一般地，编译器会根据数据类型的大小来进行自然对齐：

- char: 1 字节对齐
- short: 2 字节对齐
- int、long、float: 4 字节对齐
- double、long long: 8 字节对齐
- 指针类型：在 32 位系统下为 4 字节对齐，在 64 位系统下为 8 字节对齐 (这个我在上一题里有提到)

结构体的内存对齐规则

- 结构体的第一个成员必定在偏移量为0的地址处（PS：偏移量是指结构体/联合体（这玩意儿到目前为止我都不知道有什么用）中某个变量的起始地址相对于整个结构体起始地址的字节距离（和相对路径有点像））
- 结构体中，第二个变量开始，每个变量的起始偏移地址必须是其有效对齐值（其实就是自身大小，毕竟我用的GCC）的整数倍，如果不满足，编译器会在该变量前面填充字节
- 结构体的总大小必须是其内部**最大**有效对齐值的整数倍。如果不足，编译器会在结构体末尾补充填充字节
- 如果结构体中嵌套了另一个结构体，嵌套的结构体必须对齐到它自身内部最大对齐值的整数倍处

总而言之，关键就两个字：整数倍（呃，其实我是从0开始数的[揣手]

现在，我们来看结构类型struct PerInfo（写了这么多，我都差点儿把它忘了）：显然，数组name占10个字节（每一个char都占用一个字节），sex占1个字节，一共也就是11个字节。注意！现在age的起始地址是11不是int（4个字节）的整数倍，故而age的偏移量应当是12， $12+4=16$ ，刚好是double（8个字节）的整数倍（泰酷辣！）于是sizeof的结果应当是24，而24又恰好是double（最大数据类型）的整数倍（泰国丸美！）

有图有真相！

```
1  #include <stdio.h>
2
3  int main(){
4      typedef struct PerInfo{
5          char name[10];
6          char sex;
7          int age;
8          double height;
9      } self_information;
10
11      self_information my = {"jack", 'M', 18, 18.0};
12
13      printf("%zu", sizeof(self_information));
14
15      return 0;
16  }
```

问题 输出 调试控制台 终端 端口

5t1' '--pid=Microsoft-MIEngine-Pid-q3cniz2p.ipg' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter
=mi'
24
PS D:\c\.vscode> |

powershell
C/C++: ...
cppdbg: str...

行 16, 列 2 空格: 4 UTF-8 CRLF { } C

当然，如果我们把变量位置稍稍调换一下，就会出现不同的结果：

```
1  #include <stdio.h>
2
3  int main(){
4      typedef struct PerInfo{
5          char sex;
6          double height;
7          int age;
8          char name[10];
9      } self_information;
10
11      self_information my = {'M',18.0,18,"jack"};
12
13      printf("%zu",sizeof(self_information));
14
15      return 0;
16  }
```

问题 输出 调试控制台 终端 端口

```
stderr=Microsoft-MIEngine-Error-mtoidqah.xa1' '--pid=Microsoft-MIEngine-Pid-2kpd0151.2qr' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '-
-interpreter=mi'
32
PS D:\c\.vscode>
```

行 16, 列 2 空格: 4 UTF-8 CRLF {} C

原理已经一目了然了，言尽于此

Step2.链表操作

1.什么是链表

(1) 链表与数组的区别

- 存储
 - 链表物理存储位置是不连续的，数据分散在内存各处，其长度是随输入数据的多少而对应确定的，可以随时调整
 - 数组存储在一块连续的空间里，其长度通常在创建时就已经确定了（虽然可以用申请动态内存的方法，但是我个人觉得还是很麻烦）
- 增删改

- 链表的增加和更改相对来讲比较容易，删除也只需要修改相邻节点的指针指向
- 数组的增加和删除则需要移动大量元素，还可能涉及内存的重新分配和拷贝数据
- 访问
 - 链表的访问是顺序的（顺序存取），只能用遍历的方式
 - 数组可以通过下标直接索引

(2) 链表节点的结构特点

链表节点包含数据域和指针域：数据域用于存储元素本身，指针域用于存储下一个节点的地址，从而把各个节点按逻辑顺序连接起来。（没错，你猜对了，这是我直接copy的题干[揣手]）

根据指针域的设计不同，节点的结构特点可以分为以下三种常见类型：

- 单向链表节点：仅包含一个指向后继节点的指针域。它只能从头节点开始单向遍历，无法从后向前
- 双向链表节点：在单向链表的基础上增加了一个指向前驱节点的指针域，即每个节点都包含一个数据域和两个指针域（前驱指针与后继指针），可以双向遍历

（好像还有循环链表节点，其尾节点的指针域不再指向NULL，而是指向头节点，进而可以从任意节点开始遍历）

下面是一个只存储一个整数的单向链表节点：

```
1  typedef struct _node{
2      int value;
3      struct _node *next;
4  } node;
```

2.实现基本的链操作